

Fachhochschule Südwestfalen

Fachbereich angewandte Informatik

Modul NoSQL-Datenbanken

MongoDB als NoSQL-Datenbanken

Vorgelegt von

Malte Maximilian Krümmel

Simon Sucker

Sommersemester

2026

Warum brauchen wir neue Datenbankkonzepte?

Das ab **1970 bei IBM** entwickelte Relationenmodell organisiert Daten in Tabellen und sichert durch **ACID-Eigenschaften** einen zuverlässigen Transaktionsbetrieb – im Web-Zeitalter jedoch zunehmend ein Hindernis.

Skalierungsgrenze



Kapazitätserweiterung erfolgt fast ausschließlich durch Hardware-Ausbau. Die Verteilung jenseits 100 TB auf mehrere Maschinen erhöht den Aufwand zur Konsistenzwahrung erheblich.

Strukturelle Starre



Multistrukturelle Daten müssen vorab in ein festes Schema überführt werden — Konzeptions- und Wartungsaufwand steigen exponentiell.

Echtzeit-Performance



Das Aufteilen technischer Objekte über zahlreiche Tabellen wird problematisch, sobald komplexe Operationen in Echtzeit auszuführen sind.

Heutige Agenda

1

Grundlagen & Einordnung von NoSQL (Malte)

2

MongoDB — Datenmodell & Architektur (Malte)

3

Praxisteil (Simon)

4

Praxisteil (Simon)

5

Praxisteil (Simon)

6

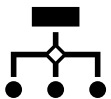
RDBMS vs. MongoDB — Vergleich & Fazit (Malte)

NoSQL-Datenbanken — Konzepte und Einordnung

Begriffsbestimmung

NoSQL = „Not only SQL“. Sammelbegriff für nicht-relationale Datenbankansätze, die parallel zu klassischen Systemen eingesetzt werden können — flexibel im Datenmodell und horizontal skalierbar.

Wesentliche Charakteristika



Horizontale Skalierbarkeit

Daten verteilt auf viele Knoten — Wachstum durch zusätzliche Maschinen statt Aufrüstung.



Hohe Verfügbarkeit

Replikation und automatisches Failover sichern den Betrieb auch bei Knotenausfällen.



Schwächeres Schemarestriktionen

Heterogene Strukturen ohne aufwändige Migrationen möglich.

Vier Grundtypen: Key-Value, Spaltenorientiert, Graph und Dokumentenorientiert

Funktionsweise und grundlegende Prinzipien



mongoDB®

(Quelle: Medium, 2026)

MongoDB ist eine **dokumentenorientierte NoSQL-Datenbank**, die Daten nicht in Zeilen und Spalten, sondern in flexiblen, JSON-ähnlichen Dokumenten speichert.

128.000+ Unternehmen weltweit setzen MongoDB ein (u. a. Adobe, eBay, Bosch)

Vier konstitutive Merkmale



Flexibles Schema

Dokumente derselben Collection dürfen unterschiedlich aufgebaut sein.



Verteilte Architektur

Replica Sets und Sharding sind nativ integriert.



MongoDB Query Language (MQL)

Mächtige Abfragesprache (MQL) inklusive Aggregation-Pipeline.



CRUD-Operationen

Create, Read, Update, Delete direkt auf Dokumentenebene.

Dokumentenorientierung und das BSON-Format

JSON (JavaScript Object Notation)

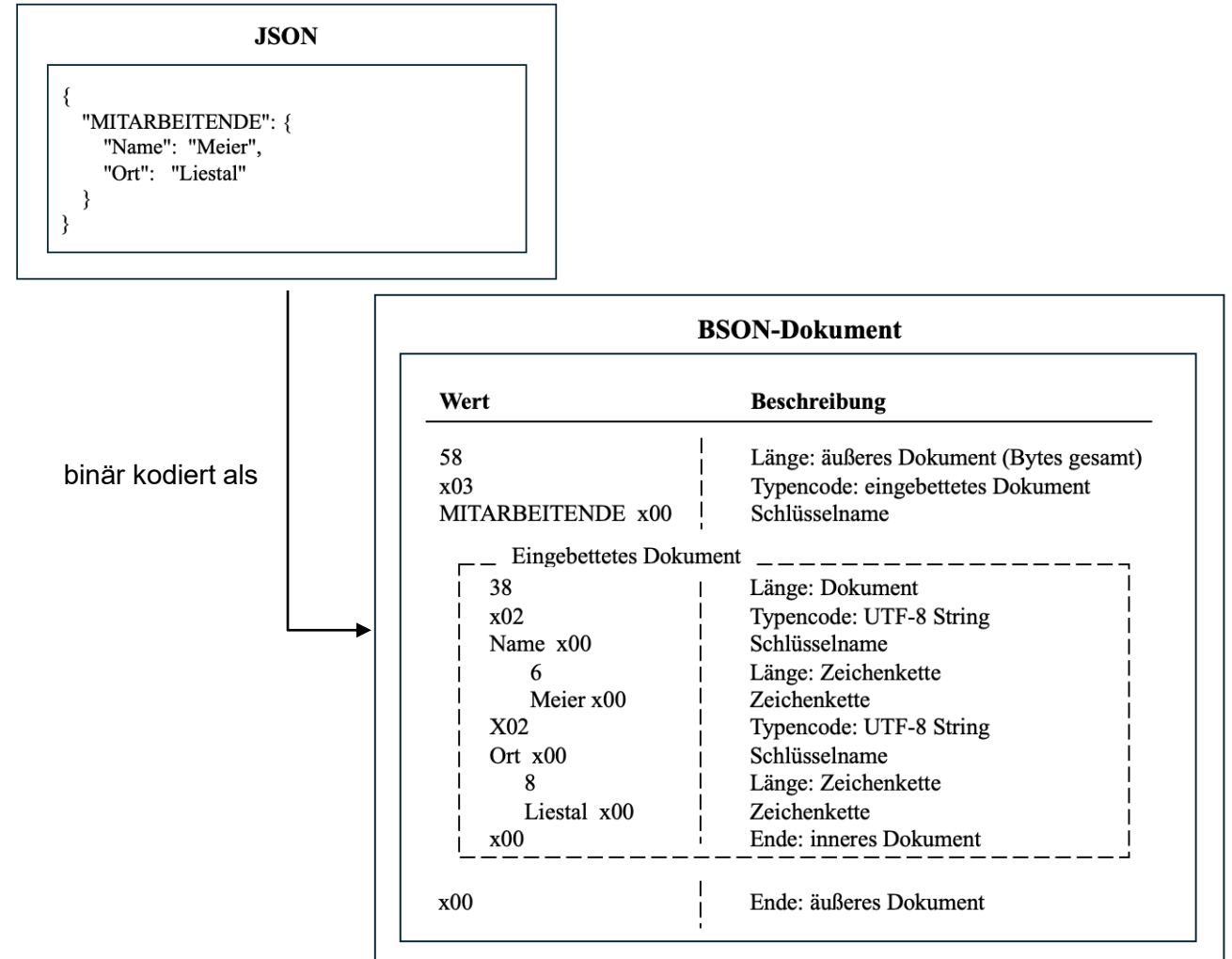
Das Austauschformat

- Textbasiert und menschenlesbar
- Weit verbreitet im Web
- Aber: langsam zu parsen
- Keine Datums- oder Binärtypen

BSON (Binary JSON)

Das Speicherformat von MongoDB

- Binär, kompakt, schnell zu lesen
- Zusätzliche Typen: Date, ObjectId, Binary, Decimal128
- Effiziente Indexierung
- Maximale Dokumentengröße: 16MB



(Quelle: Eigene Darstellung in Anlehnung an MongoDB, Inc.)

Datenmodellierung — Embedding und Referencing

Leitprinzip: Daten, auf die gemeinsam zugegriffen wird, sollten auch gemeinsam gespeichert werden.

Embedding (Einbettung)

denormalisiert

Zusammengehörige Informationen werden in einer Dokumentstruktur **verschachtelt** und können in **einer atomaren Operation** abgerufen werden.

// Collection: Kunden

```
{
  "_id": ObjectId("..."),
  "kundenname": "Anna Schmidt",
  "adresse": {
    "strasse": "Musterweg 12",
    "stadt": "Berlin",
    "plz": "10115"
  },
  "bestellung": {
    "produkt": "Laptop",
    "preis": 1299.00,
    "datum": "2026-03-01"
  }
}
```

(Quelle: Eigene Darstellung in Anlehnung an MongoDB, Inc.)

Referencing (Referenzierung)

normalisiert

Beziehungen werden über **Verweise** zwischen Collections hergestellt; die Anwendung löst die Referenzen auf.

// Collection: Kunden

```
{
  "_id": ObjectId("k001"),
  "Kundenname": "Anna Schmidt"
}
```

referenziert

// Collection: Kunden

```
{
  "_id": ObjectId("k001"),
  "kunden_id": ObjectId("k001"),
  "produkt": "Laptop",
  "preis": 1299.00
}
```

(Quelle: Eigene Darstellung in Anlehnung an MongoDB, Inc.)

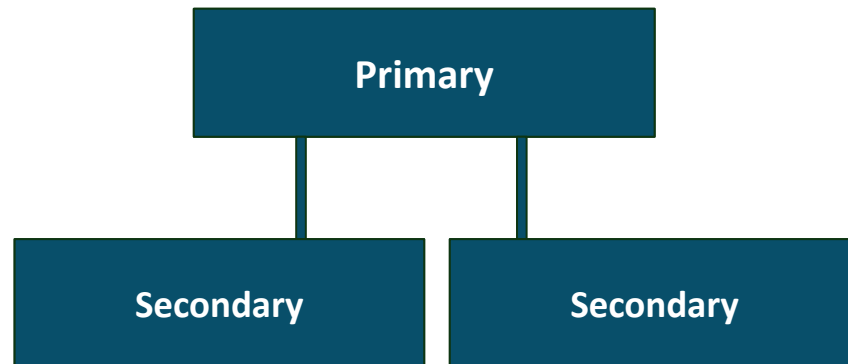
Replikation und horizontale Skalierung

Die Skalierbarkeit von MongoDB beruht auf **zwei komplementären Mechanismen**: **Replikation** gewährleistet Hochverfügbarkeit durch Redundanz, **Sharding** ermöglicht horizontale Skalierung durch Datenverteilung.



Replica Sets

Hochverfügbarkeit durch Datenreplikation



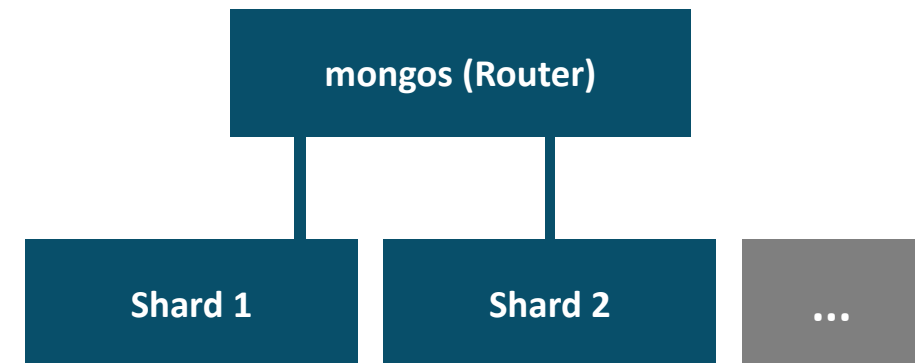
(Quelle: Eigene Darstellung in Anlehnung an MongoDB, Inc.)

Bei Ausfall des Primary wird automatisch ein Secondary zum neuen Primary gewählt (Election).



Sharding

Horizontale Skalierung durch Datenpartitionierung



(Quelle: Eigene Darstellung in Anlehnung an MongoDB, Inc.)

Daten werden basierend auf einem Shard-Key automatisch auf mehrere Server verteilt.

Konsistenz- und Transaktionsmechanismen

Kernprinzip

Schreiboperationen sind auf **Einzeldokument-Ebene atomar**. In Verbindung mit der Embedding-Strategie ist daher in den meisten Anwendungsfällen **keine explizite Transaktion** erforderlich.

Entwicklung der ACID-Unterstützung:



! Verteilte Transaktionen sind performance-aufwändig — Datenmodellierung sollte ihren Einsatz minimieren.

Mongo DB im Vergleich zu relationalen Datenbanksystemen

Dimension	RDBMS	Mongo DB
Datenmodell	Normalisierte Tabellen mit festem Schema; Beziehungen via Primär-/Fremdschlüssel.	BSON-Dokumente in Collections; flexibles, dynamisches Schema.
Skalierung	Vorrangig vertikal; ACID erschwert horizontale Verteilung.	Horizontal ausgelegt; Replica Sets und Sharding ohne praktische Grenze.
Transaktionen	ACID seit Jahrzehnten Standard; bewährt für Finanzanwendungen.	Multi-Document seit v4.0; Einsatz aus Performance-Gründen begrenzt.
Abfragesprache	SQL — breit standardisiert; ausgeprägte JOIN-Fähigkeit.	MQL — funktional vergleichbar, weniger standardisiert.

- Die Eignung hängt vom **Anforderungsprofil** der Anwendung ab — kein „besser“ oder „schlechter“, sondern „passender“.

Synthese und Fazit

Stärken von MongoDB

- ✓ **Flexibilität**
Schemaloses Dokumentenmodell passt sich Änderungen an.
- ✓ **Skalierung**
Horizontale Verteilung über Sharding ... ohne praktische Grenzen
- ✓ **Hochverfügbarkeit**
Replica Sets mit automatischem Failover.
- ✓ **Performance**
Stark bei Echtzeit-Lasten und großen Datenmengen.

Grenzen und Trade-Offs

- **Konsistenz**
Schwächer als ACID-RDBMS ... zugunsten Verfügbarkeit.
- **Transaktionen**
Verteilte Transaktionen mit Performance-Overhead.
- **Redundanz**
Embedding kann zu Datenduplikationen führen.
- **Standardisierung**
MQL weniger standardisiert



MongoDB ist **kein Ersatz** für relationale Datenbanken — sondern die **richtige Wahl**, wenn Flexibilität, Skalierung und Geschwindigkeit Priorität haben.

Agenda des Praxisteils

- Anbindung einer MongoDB Datenbank
- Konfiguration einer MongoDB Datenbank
- Vergleich von MongoDB mit PostgreSQL
 - Dokument vs Main-Entity
 - Aggregation vs Joins
 - Best Practices im Umgang mit MongoDB Systemen
 - Failover Demo

Demo-Aufbau

- Beispielanwendung: E-Commerce-Shop
- CRUD-Funktionen
- Simulation einer verteilten Systemarchitektur in MongoDB mithilfe von Docker Containern
- PostgreSQL ist auf einen Single Node Use Case angelegt
- MongoDB-Backend und PostgreSQL-Backend liefern dasselbe Produkt-JSON
- Ein gemeinsames Frontend als Vergleichsprodukt
- Dadurch ist der Vergleich fachlich fair und direkt sichtbar

Implementierung von CRUD-Operationen

- Codebeispiele

Write Concern, Read Concern und Read Preference

1. Write Concern (Schreibbestätigungen)

1. **w=1 (Bestätigung durch Primary)**

- *Vorteil:* Geringe Latenz und hohe Reaktionsfähigkeit.
- *Nachteil:* Risiko von Datenverlust bei raschem Failover ohne erfolgte Replikation.

2. **w=majority (Bestätigung der Mehrheit)**

- *Vorteil:* Erheblich reduziertes Risiko für Datenverluste.
- *Nachteil:* Höhere Latenz; Writes können bei Netzwerkpartitionen blockieren.

2. Read Concern (Lese-Isolation)

1. **local:** Sehr schnelles Lesen, aber Daten sind nicht zwingend mehrheitlich bestätigt.

2. **majority:** Stärkere Sichtbarkeitsgarantien (vermeidet das Lesen von Daten, die evtl. wieder verworfen werden).

3. Read Preference (Lese-Routing)

1. **primary:** Liest immer den aktuell führenden Stand (starke Konsistenz).

2. **secondary:** Verteilt die Leselast im Cluster, kann aber leicht veraltete Daten liefern (Eventual Consistency).

Konfigurierung der verteilten Architektur von MongoDB

- MongoDB läuft als Replica Set mit mongo1, mongo2 und Arbiter
- PostgreSQL läuft als Single-Node-Anwendung ohne Replicas / Read-Clones
- Kein Sharding, bewusst reduzierter Aufbau für den Praxisteil
- Seed-Container initialisieren die Demo-Daten für beide Datenbank-Systeme
- Statusanzeige im Showcase zeigt aktuelles Primary-Mitglied und Konfiguration

Dokumentaufbau vs Main Entity

- MongoDB speichert ein Produkt als Aggregat in einem Dokument
- Kategorien, Attribute, Varianten und Reviews liegen nahe am Produktkontext
- PostgreSQL rekonstruiert dieselbe Fachsicht aus mehreren Tabellen

Aggregation Framework vs SQL Query

- Zwei verschiedene Sprachen für das gleiche Problem: Daten auswerten

Konsequenzen eines Ausfalls / Netzwerkpartitionierung / Failover

- PostgreSQL (CP-System):
 - Einzige Node fällt aus, daher werden alle eingehenden Anfragen direkt abgelehnt. Die Verfügbarkeit (A - Availability) ist verletzt
 - Selbst wenn man Read-Clone anlegen würde, würde die Situation die gleiche bleiben, sobald der Write-Master ausfällt.
 - Dafür: Dauerhaft konsistente Daten, da keine Befehle halb oder nur auf einem Teil der Systeme ausgeführt wurden (C-Consistency)
- MongoDB (CP-System):
 - Aufgrund des Write-Concern wäre der Fall ähnlich wie bei PostgreSQL
 - Da eine Operation erst als abgeschlossen und erfolgreich gilt, wenn die Mehrheit der Knoten die Operation erreicht hat werden alle Anfragen blockiert, sobald die anderen Knoten nicht mehr erreicht werden können.
- MongoDB (AP-System):
 - Primary Node fällt aus, Secondary Node wird zur Primary gewählt
 - Durch OpLog werden in der Zwischenzeit durchgeführte anfragen später, sobald die ehemalige Primary-Node wieder online geht auf diesem nachgeholt. Je nach Konfiguration bleibt dann die neu erwählte Primary-Node oder das System wechselt zurück auf die ursprüngliche Primary-Node
 - Auch wenn die Knoten voneinander getrennt werden laufen sie weiter, da der Write-Concern nur den eigenen Knoten betrifft, der kleinere Teil kann jedoch nur noch Lesen => Hohe Verfügbarkeit, da Knoten auch unabhängig voneinander arbeiten können
 - Bei Synchronisation der Systeme aufgrund von Wiederverbindung des Netzwerks können Fehler beim Zusammenführen der Daten auftreten.

Danke für Ihre Aufmerksamkeit!
Zeit für Fragen.

Verzeichnis der Fachbücher und Ausätze in Fachzeitschriften

Autor, xx

Verzeichnis der Internetquellen

Autor, xx